LangChain: A Comprehensive Guide to Building LLM Applications

Part: 1

Instructor: Nitish Singh (CampusX)

YouTube Channel: CampusX

Playlist: LangChain Playlist

June 25, 2025

Contents

1 Components . . . 5
- 1.1 Introduction to LangChain . . . 5
- 1.2 Models . . . 5
  - 1.2.1 Types of Models . . . 5
- 1.3 Prompts . . . 6
  - 1.3.1 Types of Prompts in LangChain . . . 6
- 1.4 Chains . . . 7
- 1.5 Indexes . . . 7
- 1.6 Memory . . . 7
- 1.7 Agents . . . 8
  - 1.7.1 How Does an Agent Work? . . . 8

2 Models . . . 11
- 2.1 Introduction . . . 11
- 2.2 Model Component Overview . . . 11
- 2.3 Language Models vs. Chat Models . . . 11
- 2.4 Embedding Models . . . 12
- 2.5 Open-Source Models . . . 12
- 2.6 Setup Instructions . . . 13
- 2.7 Code Examples . . . 13
  - 2.7.1 Chat Model with Hugging Face API (TinyLlama) . . . 13
  - 2.7.2 Chat Model Locally (TinyLlama) . . . 14
  - 2.7.3 Embedding Model for Single Query . . . 14
  - 2.7.4 Embedding Model for Documents . . . 14
  - 2.7.5 Document Similarity Application . . . 14
- 2.8 Key Takeaways . . . 15
- 2.9 Next Steps . . . 15

3 Prompts . . . 17
- 3.1 Introduction . . . 17
- 3.2 Prompts Overview . . . 17
- 3.3 Static vs. Dynamic Prompts . . . 17
  - 3.3.1 Static Prompts . . . 17
  - 3.3.2 Dynamic Prompts . . . 18
- 3.4 PromptTemplate Class . . . 18
- 3.5 ChatPromptTemplate Class . . . 18
- 3.6 Message Types . . . 18
- 3.7 MessagePlaceholder . . . 18
- 3.8 Setup Instructions . . . 19

3.9 Code Examples . . . 19
3.9.1 Static Prompt with TinyLlama . . . 19
3.9.2 Dynamic Prompt with PromptTemplate . . . 20
3.9.3 Chatbot with Chat History . . . 21
3.9.4 ChatPromptTemplate . . . 21
3.9.5 MessagePlaceholder for Chat History . . . 21
3.10 Key Takeaways . . . 22
3.11 Next Steps . . . 22

4 Structured Outputs . . . 23
4.1 Introduction . . . 23
4.2 Recap of Previous Videos . . . 23
4.3 Structured vs. Unstructured Output . . . 23
4.3.1 Unstructured Output . . . 23
4.3.2 Structured Output . . . 24
4.4 Use Cases . . . 24
4.5 Structured Output in LangChain . . . 24
4.6 with_structured_output Function . . . 24
4.7 Schema Specification Methods . . . 25
4.7.1 TypedDict . . . 25
4.7.2 Pydantic . . . 25
4.7.3 JSON Schema . . . 25
4.8 Additional Features . . . 25
4.9 Setup Instructions . . . 26
4.10 Code Examples . . . 26
4.10.1 Structured Output with TypedDict . . . 26
4.10.2 Structured Output with Pydantic . . . 27
4.10.3 Structured Output with JSON Schema . . . 27
4.10.4 Pydantic Introduction . . . 28
4.11 Key Takeaways . . . 29
4.12 Next Steps . . . 29

5 Output Parsers . . . 31
5.1 Introduction . . . 31
5.2 Recap: Structured vs. Unstructured Output . . . 31
5.3 What Are Output Parsers? . . . 32
5.4 StringOutputParser . . . 32
5.4.1 Purpose . . . 32
5.4.2 When to Use . . . 32
5.4.3 Example Code . . . 32
5.5 JSONOutputParser . . . 33
5.5.1 Purpose . . . 33
5.5.2 Limitation . . . 33
5.5.3 Example Code . . . 33
5.6 StructuredOutputParser . . . 34
5.6.1 Purpose . . . 34
5.6.2 Limitation . . . 34
5.6.3 Example Code . . . 34

5.7 PydanticOutputParser . . . 35
5.7.1 Purpose . . . 35
5.7.2 Advantages . . . 35
5.7.3 Example Code . . . 35
5.8 Summary of Parsers . . . 36

6 Chains . . . 37
6.1 Recap of the LangChain Playlist So Far . . . 37
6.2 Why Chains? . . . 37
6.3 Basic Sequential Chain . . . 38
6.3.1 Imports & Setup . . . 38
6.3.2 Chain Definition . . . 38
6.3.3 Invocation Example . . . 38
6.4 More Complex Sequential Chain . . . 38
6.5 Parallel Chain . . . 39
6.5.1 Imports & Setup . . . 39
6.5.2 Define Prompts . . . 39
6.5.3 Models & Parser . . . 39
6.5.4 Build Parallel & Merge Chain . . . 40
6.5.5 Invocation . . . 40
6.6 Conditional Chain . . . 40
6.6.1 Imports & Setup . . . 40
6.6.2 Define Feedback Schema & Parser . . . 40
6.6.3 Classification Prompt . . . 41
6.6.4 Response Prompts . . . 41
6.6.5 Build Branch Chains . . . 41
6.6.6 Conditional Runnable . . . 41
6.6.7 Invocation Example . . . 42
6.7 Key Takeaways . . . 42

Chapter 1

Components

1.1 Introduction to LangChain

LangChain is an open-source framework designed to help you build applications powered by Large Language Models (LLMs), like GPT-4, with a focus on chaining components together. It's especially useful when you want to go beyond just prompting an LLM and instead build more complex, interactive, and intelligent applications, such as:

- Chatbots with memory
- Retrieval-Augmented Generation (RAG) systems
- Agents that can decide and act
- LLM-powered tools that interact with APIs, documents, or databases

1.2 Models

In LangChain, Models are the core interfaces through which you interact with AI models.

1.2.1 Types of Models

- **Language Model**: Language Models generate or complete text. They are used for:
  - Question answering
  - Summarization
  - Chatbots
  - Document generation
  - Reasoning tasks

  Popular LLM Providers:
  - OpenAI (GPT-3.5, GPT-4)
  - Anthropic (Claude)
  - HuggingFace models

  - Google PaLM
  - Cohere, AI21

- **Embedding Model**: Embedding Models convert text into numerical vectors (embeddings) that capture semantic meaning. These are used for:

  - Text similarity
  - Document retrieval
  - Clustering
  - Search

  Popular Embedding Providers:

  - OpenAI (text-embedding-ada-002)
  - HuggingFace sentence-transformers
  - Cohere embeddings
  - Google VertexAI embeddings

1.3 Prompts

A prompt is the input you send to a language model to get a desired response. It's like giving instructions to an assistant.

1.3.1 Types of Prompts in LangChain

LangChain supports several types of prompts:

1. **PromptTemplate** (Dynamic and Reusable Prompts)

   - Use placeholders to create reusable prompts.
   - Great for structured tasks like Q&A, summarization, translation.

2. **ChatPromptTemplate** (Role-Based Prompts)

   - Designed for Chat Models like gpt-3.5-turbo, using roles (system, user, assistant).
   - Encourages better multi-turn conversations.

3. **FewShotPromptTemplate**

   - Provides few-shot learning by including examples in the prompt.

1.4 Chains

A Chain is a sequence of steps that connect various components like:

- Prompts
- Language Models
- Embedding models
- Retrievers
- Tools
- Memory

Chains allow you to go beyond a single call to an LLM — they orchestrate multi-step reasoning, document retrieval, or tool usage.

1.5 Indexes

Indexes are fundamental building blocks used to organize and search over documents, especially in Retrieval-Augmented Generation (RAG) applications. Components:

1. **DocLoader**: Load your files (PDFs, CSVs, TXT, etc.).
2. **TextSplitter**: Break large documents into smaller chunks.
3. **Embeddings + VectorStore**: Convert chunks to vectors and store them for search.
4. **Retriever**: Retrieve relevant chunks at query time.

1.6 Memory

LLM APIs are stateless by default. When we say that an API is stateless, we mean that each request is independent and doesn't retain memory of previous interactions. Memory in LangChain is a component that keeps track of past interactions in a conversation and feeds them back into future prompts. This is crucial for building chatbots, agents, or multi-turn question answering systems where remembering the history is important.

Table 1.1: Memory Types in LangChain

Type	**Description**
ConversationBuffer	Stores entire conversation history as a buffer (plain text). Best for short conversations.
ConversationBuffer Window	Keeps only the last k exchanges to manage token limits.
ConversationToken Buffer	Tracks memory based on token count instead of number of messages. Helps with long conversations within token limits.

Type	Description
ConversationSummary	Uses an LLM to summarize the conversation history instead of storing the full chat. Ideal for long chats.
ConversationSummaryBuffer	Hybrid of summary and window. Keeps a summarized version of earlier chats and full recent messages.
VectorStoreRetriever	Stores and retrieves memory using vector embeddings. Great for semantic search over past messages. Useful in RAG setups.
ZepMemory WeaviateMemory RedisMemory	Integrations with external memory stores for scalable, persistent memory (Zep, Redis, etc.).

1.7 Agents

An Agent is a component that uses a language model to decide what actions to take next based on user input and its memory or tools.

1.7.1 How Does an Agent Work?

The agent:

1. Receives a user query
2. Thinks using the LLM (e.g., decides what info it needs)
3. Selects a tool (e.g., search, math, Python)
4. Uses the tool, gets results
5. Feeds the result back into the LLM
6. Repeats if needed, until it reaches a final answer

This loop is called ReAct: Reason + Act.

Table 1.2: Agent Types in LangChain

Agent Type	Description
ZeroShotAgent	Makes decisions without examples; just using the prompt.
ReActAgent	Combines reasoning (thinking steps) and acting (tool use).
ChatAgent	Built for chat models like GPT-4; integrates smoothly with ChatOpenAI.
PlanAndExecute	First plans steps, then executes them — good for complex tasks.

Agent Type	**Description**
AgentExecutor	Wrapper that runs an agent with a set of tools.

Chapter 2

Models

2.1 Introduction

This chapter summarizes the Model Component in LangChain, a framework for interacting with AI models. The focus is on using free, open-source models from Hugging Face, such as TinyLlama and Sentence Transformers, to demonstrate language and embedding model applications.

Objective: Understand the Model Component and its role in LangChain.

Purpose: Provides a unified interface for language models (text-to-text) and embedding models (text-to-vectors).

Model Types:

- **Language Models**: Input text, output text (e.g., "What is India's capital?" $\rightarrow$ "New Delhi").
- **Embedding Models**: Input text, output numerical vectors for semantic tasks.

2.2 Model Component Overview

The Model Component enables seamless interaction with AI models via a common interface, abstracting differences between providers like Hugging Face.

Key Idea: Simplifies interaction with diverse models.

Model Categories:

1. **Language Models**:
   - **LLMs**: General-purpose, deprecated in newer LangChain versions.
   - **Chat Models**: Conversational, support multi-turn dialogs and roles (system, user, assistant).
2. **Embedding Models**: Generate vectors for semantic search or Retrieval-Augmented Generation (RAG).

2.3 Language Models vs. Chat Models

Table 2.1: Comparison of LLMs and Chat Models

Aspect	**LLM**	**Chat Model**
Purpose	General NLP tasks (e.g., summarization).	Conversational tasks (e.g., chatbots).
Training	General text (books, articles).	General text + conversational fine-tuning.
Memory	No conversation history.	Supports conversation history.
Role Awareness	No role assignment.	Supports roles (e.g., system, assistant).
Examples	Older GPT-3 Instruct.	TinyLlama, Mistral (open-source).
Use Case	Text generation, translation.	Chatbots, virtual assistants.

Recommendation: Use Chat Models for modern applications.

2.4 Embedding Models

Purpose: Convert text to vectors for semantic tasks.
Use Cases: Semantic search, document similarity, RAG.
Example: Sentence Transformers (e.g., all-MiniLM-L6-v2).

2.5 Open-Source Models

Definition: Freely available models (e.g., from Hugging Face).
Advantages:

- Free, no API costs.
- Full control, local deployment.
- Data privacy, customization.

Disadvantages:

- Requires powerful hardware (e.g., GPU).
- Complex setup.
- Limited multimodal support.

Examples:

- **Language Models**: TinyLlama, Mistral.
- **Embedding Models**: Sentence Transformers.

2.6 Setup Instructions

1. Create a folder: `langchain_models`.
2. Open in VS Code.
3. Create virtual environment: `python -m venv venv`.
4. Activate:
   - Windows: `venv\Scripts\activate`
   - Linux/Mac: `source venv/bin/activate`
5. Install libraries via `requirements.txt`:
   - langchain
   - langchain-huggingface
   - python-dotenv
   - scikit-learn
   - numpy
6. Verify LangChain: `python -c "import langchain; print(langchain.__version__)"`
7. Create folders: `chat_models/`, `embedding_models/`.
8. Set Hugging Face API key in `.env`:

```
HUGGINGFACEHUB_API_TOKEN=your_huggingface_token_here
```

2.7 Code Examples

2.7.1 Chat Model with Hugging Face API (TinyLlama)

Listing 2.1: chat_model_huggingface_api.py

```python
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv

load_dotenv()
llm = HuggingFaceEndpoint(
    repo_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    task="text-generation"
)
model = ChatHuggingFace(llm=llm)
result = model.invoke("What is the capital of India?")
print(result.content)
```

2.7.2 Chat Model Locally (TinyLlama)

Listing 2.2: chat_model_huggingface_local.py

```
from langchain_huggingface import ChatHuggingFace, HuggingFacePipeline

llm = HuggingFacePipeline.from_model_id(
    model_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    task="text-generation",
    pipeline_kwargs={"temperature": 0.7, "max_new_tokens": 100}
)
model = ChatHuggingFace(llm=llm)
result = model.invoke("What is the capital of India?")
print(result.content)
```

2.7.3 Embedding Model for Single Query

Listing 2.3: embedding_hf_local_query.py

```
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
text = "Delhi is the capital of India"
embedding = embeddings.embed_query(text)
print(str(embedding))
```

2.7.4 Embedding Model for Documents

Listing 2.4: embedding_hf_local_docs.py

```
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
documents = [
    "Delhi is the capital of India",
    "Kolkata is the capital of West Bengal",
    "Paris is the capital of France"
]
doc_embeddings = embeddings.embed_documents(documents)
print(str(doc_embeddings))
```

2.7.5 Document Similarity Application

Listing 2.5: document_similarity.py

```
from langchain_huggingface import HuggingFaceEmbeddings
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

```

```
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
documents = [
    "Virat Kohli is an Indian cricketer known for his aggressive approach.",
    "Sachin Tendulkar is a cricket legend with numerous records.",
    "Jasprit Bumrah is an Indian fast bowler known for his unique action.",
    "Rohit Sharma is an Indian batsman and captain of the national team.",
    "M.S. Dhoni is a former Indian captain known for his calm demeanor."
]
doc_embeddings = embeddings.embed_documents(documents)
query = "Tell me about Virat Kohli"
query_embedding = embeddings.embed_query(query)
scores = cosine_similarity([query_embedding], doc_embeddings)[0]
scores_with_index = list(enumerate(scores))
sorted_scores = sorted(scores_with_index, key=lambda x: x[1], reverse=True)
index, score = sorted_scores[0]
print(f"Query: {query}")
print(f"Most similar document: {documents[index]}")
print(f"Similarity score: {score:.2f}")
```

Output Example:

```
Query: Tell me about Virat Kohli
Most similar document: Virat Kohli is an Indian cricketer known for his aggressive ap
Similarity score: 0.66
```

2.8 Key Takeaways

- **Model Component**: Unified interface for language and embedding models.
- **Language Models**: Chat Models are preferred over deprecated LLMs.
- **Embedding Models**: Enable semantic search and RAG.
- **Open-Source Models**: TinyLlama (chat), Sentence Transformers (embeddings) are free and accessible via Hugging Face.
- **Application**: Document similarity demonstrates semantic search.

2.9 Next Steps

- Explore Prompts in LangChain.
- Build a small chatbot application.
- Learn about vector databases and RAG.

Chapter 3

Prompts

3.1 Introduction

This chapter summarizes the Prompts Component in LangChain, focusing on designing effective prompts using free, open-source models from Hugging Face (e.g., TinyLlama). It covers static and dynamic prompts, prompt templates, chat templates, and a console-based chatbot.

Objective: Understand prompts and their role in LangChain.

Purpose: Prompts are inputs (primarily text) sent to AI models to generate responses.

Importance: Model outputs depend heavily on prompt design, leading to the emergence of Prompt Engineering.

3.2 Prompts Overview

Definition: Inputs (e.g., text) sent to AI models.

Types:

- **Static Prompts**: Fixed, hardcoded prompts.
- **Dynamic Prompts**: Contain placeholders filled at runtime.

Focus: Text-based prompts (most common).

Tools: LangChain provides `PromptTemplate` and `ChatPromptTemplate` for prompt design.

3.3 Static vs. Dynamic Prompts

3.3.1 Static Prompts

- Hardcoded (e.g., "Write a five-line poem on cricket").
- **Issues**: Inflexible, error-prone, inconsistent user experience.
- **Use Case**: Simple, one-off queries.

3.3.2 Dynamic Prompts

- Use placeholders (e.g., `{paper_title}`).
- **Benefits**: Consistency, error reduction, reusability.
- **Use Case**: Real-world applications (e.g., research tools).

3.4 PromptTemplate Class

Purpose: Creates dynamic prompts with placeholders.

Benefits:

1. Validation (e.g., `validate_template=True`).
2. Reusability (save as JSON).
3. Integration with LangChain ecosystem (e.g., chains).

vs. f-strings: `PromptTemplate` offers validation and reusability.

3.5 ChatPromptTemplate Class

Purpose: Creates dynamic prompts for multi-turn conversations.

Use Case: Dynamic system/human messages (e.g., "You are a {domain} expert").

Note: Use tuples (`role, content`) for reliable placeholder replacement.

3.6 Message Types

- **System Message**: Sets model role (e.g., "You are a helpful assistant").
- **Human Message**: User input (e.g., "Tell me about LangChain").
- **AI Message**: Model response.

Use Case: Structured chat history for context-aware conversations.

3.7 MessagePlaceholder

Definition: Placeholder for dynamically inserting chat history.

Use Case: Customer support chatbot loading past messages to answer queries like "Where is my refund?".

3.8 Setup Instructions

1. Create folder: `langchain_prompts`.
2. Open in VS Code.
3. Create virtual environment: `python -m venv venv`.
4. Activate:
   - Windows: `venv\Scripts\activate`
   - Linux/Mac: `source venv/bin/activate`
5. Install libraries via `requirements.txt`:
   - langchain
   - langchain-huggingface
   - python-dotenv
   - streamlit
6. Set Hugging Face API key in `.env`:

```
HUGGINGFACEHUB_API_TOKEN=your_huggingface_token_here
```

7. Create folders: `prompts/`, `chatbot/`.

3.9 Code Examples

3.9.1 Static Prompt with TinyLlama

Listing 3.1: static_prompt.py

```python
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv

load_dotenv()
llm = HuggingFaceEndpoint(
    repo_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    task="text-generation"
)
model = ChatHuggingFace(llm=llm)
result = model.invoke("Write a five-line poem on cricket")
print(result.content)
```

3.9.2 Dynamic Prompt with PromptTemplate

Listing 3.2: research_tool.py

```
import streamlit as st
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from langchain_core.prompts import PromptTemplate
from dotenv import load_dotenv

load_dotenv()
llm = HuggingFaceEndpoint(
    repo_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    task="text-generation"
)
model = ChatHuggingFace(llm=llm)

st.header("Research Tool")
paper_input = st.selectbox(
    "Select Paper",
    ["Attention Is All You Need", "Word2Vec", "BERT", "YOLO", "ResNet"]
)
style_input = st.selectbox(
    "Explanation Style",
    ["Simple", "Code-Heavy", "Math-Heavy"]
)
length_input = st.selectbox(
    "Explanation Length",
    ["Short", "Medium", "Long"]
)
template = """
Please summarize the research paper titled {paper_input} with the
   following specifications:

- Explanation style: {style_input}
- Explanation length: {length_input}

Include relevant mathematical equations if present in the paper.
Explain mathematical concepts using simple, intuitive code snippets
   where applicable.
Also add relatable analogies.
If any information is unavailable, state "Insufficient information
   available."
Ensure the summary is clear, accurate, and aligns with the provided
   style and length.
"""
prompt_template = PromptTemplate(
    input_variables=["paper_input", "style_input", "length_input"],
    template=template
)
if st.button("Summarize"):
    prompt = prompt_template.invoke({
        "paper_input": paper_input,
        "style_input": style_input,
        "length_input": length_input
    })
    result = model.invoke(prompt)
    st.write(result.content)
```

3.9.3 Chatbot with Chat History

Listing 3.3: chatbot.py

```
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from langchain_core.messages import SystemMessage, HumanMessage,
    AIMessage
from dotenv import load_dotenv

load_dotenv()
llm = HuggingFaceEndpoint(
    repo_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    task="text-generation"
)
model = ChatHuggingFace(llm=llm)
chat_history = [SystemMessage(content="You are a helpful AI assistant."
    )]
while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        print("Chat history:", chat_history)
        break
    chat_history.append(HumanMessage(content=user_input))
    result = model.invoke(chat_history)
    chat_history.append(AIMessage(content=result.content))
    print(f"AI: {result.content}")
```

3.9.4 ChatPromptTemplate

Listing 3.4: chat_prompt_template.py

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.messages import SystemMessage, HumanMessage

chat_template = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful {domain} expert"),
    ("human", "Explain in simple terms what is {topic}")
])
prompt = chat_template.invoke({
    "domain": "Cricket",
    "topic": "Duckworth-Lewis method"
})
print(prompt)
```

3.9.5 MessagePlaceholder for Chat History

Listing 3.5: customer_support.py

```
from langchain_core.prompts import ChatPromptTemplate,
    MessagesPlaceholder
from langchain_core.messages import SystemMessage, HumanMessage,
    AIMessage

with open("chat_history.txt", "r") as f:
    lines = f.readlines()
```

```
chat_history = [
    HumanMessage(content=lines[0].strip()),
    AIMessage(content=lines[1].strip())
]
chat_template = ChatPromptTemplate.from_messages([
    SystemMessage(content="You are a helpful customer support agent"),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{query}")
])
prompt = chat_template.invoke({
    "chat_history": chat_history,
    "query": "Where is my refund?"
})
print(prompt)
```

3.10 Key Takeaways

- **Prompts**: Critical for model interaction, sensitive to design.
- **Static Prompts**: Inflexible, error-prone.
- **Dynamic Prompts**: Use `PromptTemplate` and `ChatPromptTemplate`.
- **Message Types**: System, Human, AI for structured conversations.
- **MessagePlaceholder**: Simplifies chat history integration.
- **Open-Source**: TinyLlama ensures cost-free applications.

3.11 Next Steps

- Explore Chains in LangChain.
- Prompt Engineering playlist (e.g., Chain-of-Thought, Few-Shot Prompting).

Chapter 4

Structured Outputs

4.1 Introduction

This chapter summarizes the Structured Output concept in LangChain, focusing on generating structured data using the free, open-source model Mistral-7B-Instruct-v0.3 from Hugging Face. It covers unstructured vs. structured outputs, use cases, and the `with_structured_output` function with TypedDict, Pydantic, and JSON schema.

Objective: Generate structured outputs for integration with databases, APIs, and agents.

Purpose: Convert text responses into data formats (e.g., JSON).

Model: Mistral-7B-Instruct-v0.3 supports structured output via JSON mode.

Importance: Essential for real-world applications, especially agents.

4.2 Recap of Previous Videos

- **Video 1**: Introduction to LangChain.
- **Video 2**: Six components, including agents.
- **Video 3**: Models (LLMs, embeddings) using Hugging Face.
- **Video 4**: Prompts (static/dynamic, chat templates).
- **Video 5**: Structured output for programmatic processing.

4.3 Structured vs. Unstructured Output

4.3.1 Unstructured Output

- **Definition**: Textual responses (e.g., "New Delhi is the capital of India").
- **Issues**: No structure, hard to process programmatically.
- **Example**: Prompt: "Create a one-day travel itinerary for Paris." Response: "Morning: Visit Eiffel Tower, Afternoon: Visit a museum, Evening: Have dinner."

4.3.2 Structured Output

- **Definition**: Defined format (e.g., JSON).
- **Example**: Same prompt returns:

```
[
    {"time": "Morning", "activity": "Visit Eiffel Tower"},
    {"time": "Afternoon", "activity": "Visit a museum"},
    {"time": "Evening", "activity": "Have dinner"}
]
```

- **LangChain Definition**: Well-defined data format for parsing.

Benefits: Enables integration with databases, APIs, tools; reduces errors.

4.4 Use Cases

1. **Data Extraction**: Extract resume data for database storage (e.g., Naukri.com).
2. **API Building**: Extract review topics, pros, cons, sentiment for APIs (e.g., Amazon).
3. **Building Agents**: Extract structured data for tools (e.g., calculator).

4.5 Structured Output in LangChain

LLM Types:

- **Support Structured Output**: Mistral-7B-Instruct-v0.3 (JSON mode).
- **No Structured Output**: TinyLlama (requires output parsers).

Tools:

- `with_structured_output`: For supported models.
- **Output parsers**: For unsupported models (covered in next chapter).

Focus: `with_structured_output` with Mistral-7B-Instruct-v0.3.

4.6 with_structured_output Function

Purpose: Generates structured outputs by specifying a schema.
Workflow:

1. Define schema.
2. Call `model.with_structured_output(schema, method="json_mode").invoke(prompt)`.

Schema Methods:

- **TypedDict**: Python type hints.
- **Pydantic**: Data validation.
- **JSON Schema**: Cross-language compatibility.

Behind the Scenes: Generates system prompt (e.g., "Return JSON with schema").

4.7 Schema Specification Methods

4.7.1 TypedDict

- **Definition**: Uses `typing.TypedDict` for dictionary structure.
- **Limitations**: No runtime validation.
- **Use Case**: Simple Python projects.

4.7.2 Pydantic

- **Definition**: Library for data validation.
- **Features**: Runtime validation, defaults, optional fields, type coercion, validators.
- **Use Case**: Python projects needing validation.

4.7.3 JSON Schema

- **Definition**: JSON-based universal format.
- **Features**: Cross-language compatibility.
- **Use Case**: Multi-language projects.

Recommendation: Pydantic for Python-based LangChain projects.

4.8 Additional Features

Method Parameter:

- **JSON Mode**: Returns JSON (used with Mistral).
- **Function Calling**: For tool invocation (agents).

Model Compatibility:

- **Mistral-7B-Instruct-v0.3**: JSON mode.
- **TinyLlama**: Output parsers.

4.9 Setup Instructions

1. Create folder: `langchain_structured_output`.
2. Open in VS Code.
3. Create virtual environment: `python -m venv venv`.
4. Activate:
   - Windows: `venv\Scripts\activate`
   - Linux/Mac: `source venv/bin/activate`
5. Install libraries via `requirements.txt`:
   - langchain
   - langchain-huggingface
   - python-dotenv
   - pydantic
6. Set Hugging Face API key in `.env`:

```
HUGGINGFACEHUB_API_TOKEN=your_huggingface_token_here
```

7. Create folder: `structured_output/`.

4.10 Code Examples

4.10.1 Structured Output with TypedDict

Listing 4.1: with_structured_output_typeddict.py

```python
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv
from typing import TypedDict, Annotated

load_dotenv()
llm = HuggingFaceEndpoint(
    repo_id="mistralai/Mixtral-8x7B-Instruct-v0.1",
    task="text-generation"
)
model = ChatHuggingFace(llm=llm)

class Review(TypedDict):
    summary: Annotated[str, "A brief summary of the review"]
    sentiment: Annotated[str, "Overall sentiment: Positive, Negative or
        Neutral"]

structured_model = model.with_structured_output(Review, method="
    json_mode")
review_text = "This phone has a great battery and fast processor but
    poor camera."
```

```
result = structured_model.invoke(review_text)
print(result)
print(result["summary"])
print(result["sentiment"])
```

4.10.2 Structured Output with Pydantic

Listing 4.2: with_structured_output_pydantic.py

```
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv
from pydantic import BaseModel, Field
from typing import List, Optional, Literal

load_dotenv()
llm = HuggingFaceEndpoint(
    repo_id="mistralai/Mixtral-8x7B-Instruct-v0.1",
    task="text-generation"
)
model = ChatHuggingFace(llm=llm)

class Review(BaseModel):
    themes: List[str] = Field(description="List of key themes discussed
        in the review")
    summary: str = Field(description="A brief summary of the review")
    sentiment: Literal["Positive", "Negative"] = Field(description="
       Overall sentiment")
    pros: Optional[List[str]] = Field(default=None, description="List
       of pros")
    cons: Optional[List[str]] = Field(default=None, description="List
       of cons")
    name: Optional[str] = Field(default=None, description="Name of the
       reviewer")

structured_model = model.with_structured_output(Review, method="
   json_mode")
review_text = "This phone has a Snapdragon processor, 5000 mAh battery,
    and 45W charging. The display is vibrant, but the camera struggles
   in low light. Reviewed by Nitish Singh."
result = structured_model.invoke(review_text)
print(result)
print(result.summary)
print(result.sentiment)
print(result.name)
```

4.10.3 Structured Output with JSON Schema

Listing 4.3: with_structured_output_json.py

```
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv

load_dotenv()
llm = HuggingFaceEndpoint(
    repo_id="mistralai/Mixtral-8x7B-Instruct-v0.1",
```

```
    task="text-generation"
)
model = ChatHuggingFace(llm=llm)

json_schema = {
    "title": "Review",
    "type": "object",
    "properties": {
        "themes": {
            "type": "array",
            "items": {"type": "string"},
            "description": "List of key themes discussed in the review"
        },
        "summary": {
            "type": "string",
            "description": "A brief summary of the review"
        },
        "sentiment": {
            "type": "string",
            "enum": ["Positive", "Negative"],
            "description": "Overall sentiment: Positive or Negative"
        },
        "pros": {
            "type": ["array", "null"],
            "items": {"type": "string"},
            "description": "List of pros mentioned in the review"
        },
        "cons": {
            "type": ["array", "null"],
            "items": {"type": "string"},
            "description": "List of cons mentioned in the review"
        },
        "name": {
            "type": ["string", "null"],
            "description": "Name of the reviewer"
        }
    },
    "required": ["themes", "summary", "sentiment"]
}

structured_model = model.with_structured_output(json_schema, method="
    json_mode")
review_text = "This phone has a Snapdragon processor, 5000mAh battery,
    and 45W charging. The display is vibrant, but the camera struggles
    in low light. Reviewed by Nitish Singh."
result = structured_model.invoke(review_text)
print(result)
print(result["summary"])
print(result["sentiment"])
print(result["name"])
```

4.10.4 Pydantic Introduction

Listing 4.4: pydantic_intro.py

```
from pydantic import BaseModel, Field, EmailStr
from typing import Optional
```

```python

class Student(BaseModel):
    name: str = Field(default="Nitish", description="Name of the
        student")
    age: Optional[int] = Field(default=None, description="Age of the
        student")
    cgpa: float = Field(default=5.0, ge=0, le=10, description="Decimal
        value representing CGPA")
    email: EmailStr = Field(description="Email address of the student")

# Valid data
student = Student(
    name="Nitish",
    age="32",
    cgpa=8.5,
    email="nitish@gmail.com"
)
print(student)
print(student.dict())
print(student.age)  # Type coercion: "32" to 32

# Invalid data (will raise error)
# student = Student(name=32, age="32", cgpa=12, email="abc")
```

4.11 Key Takeaways

- **Structured Output**: Converts text into data formats (e.g., JSON).
- **Benefits**: Enables database storage, API building, agent tools.
- **Tools**: `with_structured_output` for Mistral; parsers for others.
- **Schema Methods**: TypedDict, Pydantic, JSON Schema.
- **Model**: Mistral-7B-Instruct-v0.3 for free structured output.

4.12 Next Steps

- Next Video: Output parsers for unsupported models.
- Playlist: Agents and advanced LangChain topics.

Chapter 5

Output Parsers

5.1 Introduction

This chapter summarizes the video transcript on LangChain output parsers, detailing the four most important parsers and including all code examples.

Presenter: Nitish Singh

Context: This chapter builds on the previous chapter about structured outputs in LangChain.

Objective: Explain what output parsers are, why they're needed, and cover the four most important output parsers in LangChain:

- StringOutputParser
- JSONOutputParser
- StructuredOutputParser
- PydanticOutputParser

Use Cases: Converting raw LLM responses (unstructured) into structured formats (JSON, CSV, Pydantic models) for databases or APIs.

5.2 Recap: Structured vs. Unstructured Output

- **Unstructured Output**: LLM returns plain text with metadata; not directly consumable by APIs or databases.
- **Structured Output**: LLM is forced to return output in a specific schema (e.g., JSON), making it easy to integrate.
- **Model Types**:
  - **Can** models: Fine-tuned to return structured output by default (e.g., GPT-4 with function calling).
  - **Can't** models: Open-source models not fine-tuned for structure (e.g., TinyLlama).
- **Solution**: Use LangChain output parsers to enforce structure even on 'can't' models.

5.3 What Are Output Parsers?

- **Definition**: Classes in LangChain that convert raw LLM responses into structured formats (JSON, CSV, Pydantic objects, etc.), ensuring consistency and easing integration.
- **Flexibility**: Parsers can be used with both 'can' and 'can't' models.
- **General Flow**:
  - Form a prompt (possibly with PromptTemplate).
  - Invoke LLM to get a raw response.
  - Pass the response through an output parser to extract/convert the content.
  - (Optional) Chain multiple steps together via Chain.

5.4 StringOutputParser

5.4.1 Purpose

- Simplest parser: extracts the text content of the LLM response directly as a string.
- Use when you just need the raw text without metadata.

5.4.2 When to Use

- Multi-step interactions where you take the LLM's text output and feed it back in as input.
- Scenarios where `response.content` feels verbose to type repeatedly.

5.4.3 Example Code

Listing 5.1: StringOutputParser Example

```
from langchain import HuggingFacePipeline
from transformers import pipeline
from langchain.llms import LLM  # or ChatHuggingFace
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain
from langchain.output_parsers import StringOutputParser

# Initialize model (example: TinyLlama via HuggingFace)
llm = LLM(model_name="tii/llama-1b")

# Define prompts
template1 = PromptTemplate(
    input_variables=["topic"],
    template="Write a detailed report on {topic}."
)
template2 = PromptTemplate(
    input_variables=["text"],
    template="Summarize the following text in five lines: \n{text}"
```

```
)

# Create parser
parser = StringOutputParser()

# Build chain
def build_chain():
    # Steps: prompt1 -> llm -> parser -> prompt2 -> llm -> parser
    chain = LLMChain(
        llm=llm,
        prompt=template1,
        output_parser=parser
    )
    return chain

# Manual two-step example
res1 = llm.invoke("Write a detailed report on black holes.")
report = parser.parse(res1.content)
res2 = template2.invoke({"text": report})
summary = parser.parse(res2)
print(summary)
```

5.5 JSONOutputParser

5.5.1 Purpose

- Forces the LLM to output valid JSON.
- Returns a Python `dict` by parsing the JSON string.

5.5.2 Limitation

- No schema enforcement: LLM decides JSON keys/structure.

5.5.3 Example Code

Listing 5.2: JSONOutputParser Example

```
from langchain.output_parsers import JSONOutputParser
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain

# Define template and parser
template = PromptTemplate(
    template=(
        """
        Give me the name and city of a fictional person. \n
        Return a JSON object.
        """
    )
)
parser = JSONOutputParser()

# Build and invoke
```

```
chain = LLMChain(llm=llm, prompt=template, output_parser=parser)
result = chain.invoke({})  # empty dict if no inputs
print(result['name'], result['city'])
```

5.6 StructuredOutputParser

5.6.1 Purpose

- Enforces a predefined JSON schema (field names) in LLM output.
- Returns parsed data matching the schema.

5.6.2 Limitation

- No data validation: Cannot enforce types or constraints.

5.6.3 Example Code

Listing 5.3: StructuredOutputParser Example

```
from langchain.output_parsers import StructuredOutputParser,
    ResponseSchema
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain

# Define schemas for three facts
schemas = [
    ResponseSchema(
        name="fact_one",
        description="First fact about the topic"
    ),
    ResponseSchema(
        name="fact_two",
        description="Second fact about the topic"
    ),
    ResponseSchema(
        name="fact_three",
        description="Third fact about the topic"
    )
]
parser = StructuredOutputParser.from_response_schemas(schemas)

# Define prompt
template = PromptTemplate(
    input_variables=["topic"],
    template=(
        """
        Give three facts about {topic}. \n
        Output must follow the JSON schema: \n
        {format_instructions}
        """
    )
)
```

```
# Build and invoke chain
chain = LLMChain(llm=llm, prompt=template, output_parser=parser)
result = chain.invoke({"topic": "black holes"})
print(result)
```

5.7 PydanticOutputParser

5.7.1 Purpose

- Extends StructuredOutputParser with Pydantic for full schema and data validation.
- Enforces types, value constraints.

5.7.2 Advantages

1. Strict schema enforcement
2. Type safety and conversions
3. Data validation (e.g., age $>$ 2)
4. Seamless integration with other components

5.7.3 Example Code

Listing 5.4: PydanticOutputParser Example

```
from langchain.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain

# Define Pydantic model
class Person(BaseModel):
    name: str = Field(description="Name of the person")
    age: int = Field(description="Age of the person", gt=2)
    city: str = Field(description="City of residence")

parser = PydanticOutputParser(pydantic_object=Person)

# Define template
template = PromptTemplate(
    input_variables=["country"],
    template=(
        """
        Generate name, age, and city for a fictional {country} person.
            \n
        {format_instructions}
        """
    )
)

# Build and invoke chain
chain = LLMChain(llm=llm, prompt=template, output_parser=parser)
```

```
result: Person = chain.invoke({"country": "Indian"})
print(result.name, result.age, result.city)
```

5.8 Summary of Parsers

Table 5.1: Summary of LangChain Output Parsers

Parser	**Output Type**	**Schema Enforcement**	**Validation**	**Typical Use Case**
String	str	No	No	Simple text extraction
JSON	dict	No	No	Quick JSON from any LLM
Structured	dict	Yes (fields only)	No	Enforce field schema but no type checks
Pydantic	Pydantic Model	Yes + Validation	Yes	Full schema + validation, complex pipelines

Key Takeaway: LangChain output parsers enable you to build robust, multi-step chains that produce reliable structured data from arbitrary LLMs, integrating seamlessly with APIs, databases, and downstream systems.

Chapter 6

Chains

6.1 Recap of the LangChain Playlist So Far

- **Models Component**: Learned how to interact with different AI/LLM models via LangChain.
- **Prompts Component**: Designed various prompt templates and sent inputs to LLMs.
- **Structured Output & Parsers**: Generated structured outputs (JSON, data-classes, etc.) and parsed them with LangChain's output parsers.

6.2 Why Chains?

Every LLM-based app consists of small steps:

1. Prompt design
2. Invoke LLM
3. Process response

Managing each step manually gets tedious as complexity grows. Chains let you link these steps into a pipeline:

$$\text{Input} \rightarrow \text{PromptTemplate} \rightarrow \text{Model.invoke} \rightarrow \text{OutputParser} \rightarrow \text{FinalResult}$$

- **Automatic data flow**: Output of step 1 feeds step 2, and so on.
- **Trigger once**: Supply only the first input and call `chain.invoke(...)` to run all steps.
- **Flexible structures**: Besides linear (sequential), you can build parallel, conditional, or custom-shaped pipelines.

6.3 Basic Sequential Chain

6.3.1 Imports & Setup

Listing 6.1: Basic Sequential Chain Setup

```python
from langchain.chat_models import ChatOpenAI
from langchain.prompts import PromptTemplate
from langchain.output_parsers import StringOutputParser

# Initialize components
prompt = PromptTemplate.from_template(
    "Generate five interesting facts about the topic: {topic}"
)
model = ChatOpenAI()  # default model
parser = StringOutputParser()
```

6.3.2 Chain Definition

```python
# Chain = prompt -> model -> parser
chain = prompt | model | parser
```

6.3.3 Invocation Example

```python
result = chain.invoke({"topic": "cricket"})
print(result)
```

The entire flow—template filling, model call, parsing—runs in one line.

6.4 More Complex Sequential Chain

Generate a detailed report, then summarize its five key points.

Listing 6.2: Complex Sequential Chain

```python
from langchain.prompts import PromptTemplate
from langchain.chat_models import ChatOpenAI
from langchain.output_parsers import StringOutputParser

# Prompt #1: Detailed report
template1 = PromptTemplate.from_template(
    "Generate a detailed report on the topic: {topic}"
)

# Prompt #2: Five-point summary
template2 = PromptTemplate.from_template(
    "From the following text generate a five-point summary: \n\n{text}"
)

model = ChatOpenAI()
parser = StringOutputParser()

# Build chain: report -> summary
```

```
report_chain = template1 | model | parser
summary_chain = template2 | model | parser
full_chain = report_chain | summary_chain

# Invoke
topic = "Employment trends in India"
result = full_chain.invoke({"topic": topic})
print(result)
```

6.5 Parallel Chain

Build notes and quiz from the same input document in parallel, then merge.

6.5.1 Imports & Setup

Listing 6.3: Parallel Chain Imports

```
from langchain.chat_models import ChatOpenAI
from langchain.chat_models import ChatAnthropic
from langchain.prompts import PromptTemplate
from langchain.output_parsers import StringOutputParser
from langchain.schema import RunnableParallel
```

6.5.2 Define Prompts

```
# Notes prompt
notes_prompt = PromptTemplate.from_template(
    "Generate short and simple notes from the following text: \n\n{text
       }"
)

# Quiz prompt
quiz_prompt = PromptTemplate.from_template(
    "Generate five short question-answers from the following text: \n\n
       {text}"
)

# Merge prompt
merge_prompt = PromptTemplate.from_template(
    "Merge the provided notes and quiz into a single document: \n\
       nNotes: \n{notes} \n\nQuiz: \n{quiz}"
)
```

6.5.3 Models & Parser

```
model1 = ChatOpenAI()  # for notes
model2 = ChatAnthropic(model_name="claude-3")  # for quiz
parser = StringOutputParser()
```

6.5.4 Build Parallel & Merge Chain

```
# Sequential chains for each branch
notes_chain = notes_prompt | model1 | parser
quiz_chain = quiz_prompt | model2 | parser

# Parallel runnable
parallel = RunnableParallel({
    "notes": notes_chain,
    "quiz": quiz_chain
})

# Merge chain
merge_chain = merge_prompt | model1 | parser

# Full pipeline: parallel -> merge
full_chain = parallel | merge_chain
```

6.5.5 Invocation

```
text = """
Linear regression is a supervised learning algorithm...
(large document on ML)
"""
result = full_chain.invoke({"text": text})
print(result)
```

6.6 Conditional Chain

Run different chains based on sentiment classification.

6.6.1 Imports & Setup

Listing 6.4: Conditional Chain Imports

```
from langchain.chat_models import ChatOpenAI
from langchain.prompts import PromptTemplate
from langchain.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field
from typing import Literal
from langchain.schema import RunnableBranch, RunnableLambda
```

6.6.2 Define Feedback Schema & Parser

```
class Feedback(BaseModel):
    sentiment: Literal["positive", "negative"] = Field(
        description="Sentiment of the feedback"
    )

sentiment_parser = PydanticOutputParser(pydantic_object=Feedback)
```

6.6.3 Classification Prompt

```
classify_prompt = PromptTemplate.from_template(
    "Classify the sentiment of the following feedback into positive or
       negative: \n\n{feedback}"
)
```

6.6.4 Response Prompts

```
# Positive response
positive_prompt = PromptTemplate.from_template(
    "Write a warm thank-you reply for positive feedback: \n\n{feedback}
       "
)

# Negative response
negative_prompt = PromptTemplate.from_template(
    "Write an empathetic apology reply for negative feedback: \n\n{
       feedback}"
)
```

6.6.5 Build Branch Chains

```
model = ChatOpenAI()

# Classification chain
classify_chain = classify_prompt | model | sentiment_parser

# Positive branch
pos_chain = positive_prompt | model | StringOutputParser()

# Negative branch
neg_chain = negative_prompt | model | StringOutputParser()

# Default (lambda) branch
default_chain = RunnableLambda(lambda x: "Could not determine sentiment
   .")
```

6.6.6 Conditional Runnable

```
# Conditional runnable
branch = RunnableBranch(
    (
        lambda x: x.sentiment == "positive",
        pos_chain
    ),
    (
        lambda x: x.sentiment == "negative",
        neg_chain
    ),
    default_chain
)
```

```

# Final pipeline: classify -> branch
final_chain = classify_chain | branch
```

6.6.7 Invocation Example

```
feedback = "This is a terrible phone."
result = final_chain.invoke({"feedback": feedback})
print(result)
```

6.7 Key Takeaways

1. **Sequential Chains** for linear pipelines.
2. **Parallel Chains** to execute multiple subtasks concurrently.
3. **Conditional Chains** (RunnableBranch) to route logic like if-else.

These patterns form the backbone of any complex LangChain application—and as you move on to Agents and Tools, you'll see how chains underlie them all.